SUPPLY CHAIN INVENTORY

SQL Data Cleaning & Quality Analysis

Complete MySQL Project Documentation

Tool	Raw Rows	Clean Rows	Issues Fixed
MySQL 8.x	**105**	**~72**	**13**

GitHub: github.com/tensakuro/mysql-files

1. Introduction

Data quality is the foundation of any reliable data-driven system. In supply chain management, even small errors — a wrong phone number, a negative stock figure, or an inconsistent date format — can cascade into faulty reports, incorrect reorder triggers, and poor business decisions. This project presents a complete SQL-based data quality pipeline applied to a supply chain inventory dataset, simulating real-world data engineering practice.

Key Concept

What is Data Cleaning?
Data cleaning (also called data cleansing or data scrubbing) is the process of detecting and correcting
corrupt, inaccurate, or irrelevant records from a dataset. It involves handling missing values, removing duplicates, standardizing formats, fixing data types, and enforcing business rules. Poor data quality is widely recognised as one of the most expensive problems in enterprise systems (Rahm & Do, 2000).

1.1 Project Objectives

- Identify every data quality issue in the raw supply table through systematic SQL auditing
- Apply targeted SQL transformations to resolve each issue with documented reasoning
- Produce a validated, constraint-enforced clean_inventory table ready for reporting
- Demonstrate reusable SQL patterns applicable to real-world data engineering workflows

1.2 Dataset Overview

The source table supply contains 13 columns and 105 rows representing grocery and spice products across three warehouses. All columns were imported as VARCHAR(50) — meaning numeric and date fields are stored as plain text — which is a deliberate design choice to avoid silent type-conversion errors at load time and instead catch them explicitly during cleaning.

Column	Raw Type	Description / Data Quality Risk
`product_id`	`VARCHAR(50)`	Unique product code (e.g. P001) — may have duplicates
`product_name`	`VARCHAR(50)`	Product name — may be blank, '???', or 'unknown'
`category`	`VARCHAR(50)`	Category label — many inconsistent variants found
`supplier_name`	`VARCHAR(50)`	Supplier name — may contain special chars or be blank
`supplier_contact`	`VARCHAR(50)`	10-digit Indian mobile — may be letters, too short, negative
`warehouse_location`	`VARCHAR(50)`	Warehouse — stored inconsistently (wh-a, WH B, etc.)
`quantity_in_stock`	`VARCHAR(50)`	Integer — stored as text; may be negative, zero, or words
`reorder_level`	`VARCHAR(50)`	Integer threshold — same risks as quantity_in_stock
`unit_price`	`VARCHAR(50)`	Integer price — may be 'expensive', negative, or blank
`last_restocked_date`	`VARCHAR(50)`	Date — three different formats found in same column
`expiry_date`	`VARCHAR(50)`	Date — same mixed-format problem; may precede restock date
`batch_number`	`VARCHAR(50)`	Batch identifier — may be blank or NULL
`status`	`VARCHAR(50)`	Active/Inactive/Pending — many non-standard variants

2. Schema Exploration

2.1 Describe Table Structure

The first step in any data project is understanding what you are working with. The DESC command reveals each column's name, data type, whether it accepts NULL, and any default values.

```
DESC supply;
```

Why this query?

DESC (short for DESCRIBE) is used before writing any cleaning logic because it tells us:
- What columns exist and how they are named — avoids typos in subsequent queries
- Every column is VARCHAR(50) — confirms all numeric/date fields need type validation
- All columns allow NULL — means the database enforces no completeness rules at all

Understanding the schema upfront prevents wasted effort on columns that don't exist or are already constrained by the database itself.

2.2 Row Count Baseline

Before any transformation, record the total number of rows. This baseline is used at the end to calculate how many rows were removed or retained, proving that the cleaning pipeline did not silently drop good data.

```
SELECT COUNT(*) AS no_of_rows
FROM supply;

-- Result: 105 rows
```

Why this query?

Capturing the pre-cleaning row count is a data engineering best practice. It lets you verify at the end that the number of rows removed matches the number of known issues. Here: 105 raw -> ~72 clean. Without this baseline, you cannot tell whether rows were dropped correctly or accidentally.

3. Data Quality Audit

The audit phase identifies every quality problem before touching a single row. This is critical — if you start fixing issues before fully mapping them, you may accidentally hide problems or apply fixes in the wrong order. All findings below were discovered before any DELETE or UPDATE was executed.

3.1 Duplicate product_id Values

A product_id should uniquely identify each product. Duplicates mean two rows claim to be the same product, which corrupts joins, counts, and any aggregation that assumes uniqueness.

```
-- Method A: Window function — shows all duplicate rows in full
```

```
SELECT * FROM (
  SELECT *, COUNT(*) OVER (PARTITION BY product_id) AS cnt
  FROM supply
) t
WHERE cnt > 1;

-- Method B: GROUP BY / HAVING — simpler, just lists which IDs are
duplicated
SELECT product_id, COUNT(*)
FROM supply
GROUP BY product_id
HAVING COUNT(*) > 1;

-- Result: P001 x2,  P003 x2,  P005 x2
```

Why this query?

Two complementary methods are shown deliberately:

- Window function (COUNT OVER PARTITION BY): Returns the full row for every duplicate, so you can inspect all columns and decide which copy to keep. Useful when rows differ in other fields.
- GROUP BY / HAVING COUNT(*) > 1: Faster and simpler — just tells you which product_ids repeat. Use this first for a quick count, then switch to the window function to inspect differences.

PARTITION BY product_id restarts the count for each unique ID; the outer WHERE then filters only rows where that count exceeded 1. This is more elegant than a self-join and keeps all row data visible.

3.2 Null and Blank Value Counts per Column

MySQL distinguishes between NULL (absence of a value) and an empty string '' (a value that exists but is blank). Both are equally problematic in a clean dataset, so we check for both simultaneously across all 13 columns in a single query.

```
SELECT
  SUM(product_id IS NULL OR TRIM(product_id) = '')           AS
null_product_id,
  SUM(product_name IS NULL OR TRIM(product_name) = '')        AS
null_product_name,
  SUM(category IS NULL OR TRIM(category) = '')                AS
null_category,
  SUM(supplier_name IS NULL OR TRIM(supplier_name) = '')      AS
null_supplier_name,
  SUM(supplier_contact IS NULL OR TRIM(supplier_contact)='')  AS
null_supplier_contact,
  SUM(warehouse_location IS NULL OR TRIM(warehouse_location)='') AS
null_warehouse,
  SUM(quantity_in_stock IS NULL OR TRIM(quantity_in_stock)='') AS
null_quantity,
  SUM(reorder_level IS NULL OR TRIM(reorder_level)='')        AS
null_reorder_level,
```

```
  SUM(unit_price IS NULL OR TRIM(unit_price)='')              AS
null_unit_price,
  SUM(last_restocked_date IS NULL OR TRIM(last_restocked_date)='') AS
null_restock_date,
  SUM(expiry_date IS NULL OR TRIM(expiry_date)='')            AS
null_expiry_date,
  SUM(batch_number IS NULL OR TRIM(batch_number)='')          AS
null_batch_number,
  SUM(status IS NULL OR TRIM(status)='')                      AS null_status
FROM supply;

-- Results (selected): product_name=4, category=4, supplier_contact=4,
--   warehouse_location=4, quantity_in_stock=6, reorder_level=6,
unit_price=6
```

Why this query?

SUM(condition) works because MySQL evaluates boolean expressions as 1 (TRUE) or 0 (FALSE). Summing them gives the total count of rows where the condition is true — equivalent to COUNT(*) with
a WHERE clause, but all columns are checked in a single table scan (much more efficient).

TRIM() is applied before the empty-string check to catch values that are just spaces (' '). Without TRIM, a cell containing ' ' would pass the = '' check and be missed as a null.

Doing all 13 columns in one query creates a 'null profile' of the entire dataset at a glance.

3.3 Non-Numeric Values in Numeric Columns

Because quantity_in_stock, unit_price, and reorder_level were imported as VARCHAR, any cell can contain arbitrary text. Real examples found: 'abc', 'expensive', 'two hundred', 'fifty five', 'bad'. These rows cannot be converted to numbers and must be removed.

```
SELECT product_id, product_name, quantity_in_stock, unit_price,
reorder_level
FROM supply
WHERE quantity_in_stock NOT REGEXP '^[0-9]+$'
   OR unit_price        NOT REGEXP '^[0-9]+$'
   OR reorder_level     NOT REGEXP '^[0-9]+$';

-- Examples found: 'abc', 'expensive', 'two hundred', 'fifty five', 'bad'
```

Why this query?

REGEXP '^[0-9]+$' uses a regular expression to test whether a string contains ONLY digits:

- ^ anchors to the start of the string
- [0-9]+ means one or more digit characters (0 through 9)
- $ anchors to the end of the string

NOT REGEXP finds every value that fails this pattern — any value containing letters or symbols.

Note: '^[0-9]+$' (no minus sign) is used intentionally here — negatives are a separate category of issue handled in section 3.4. Keeping the two problems distinct makes the audit clearer.

3.4 Negative Values in Numeric Columns

Stock quantities, reorder levels, and prices cannot logically be negative. These values indicate data entry errors. They are identified separately from non-numeric text because the fix is different — these rows contain valid-looking numbers but violate business rules.

```
SELECT product_id, quantity_in_stock, reorder_level, unit_price
FROM supply
WHERE TRIM(quantity_in_stock) LIKE '-%'
   OR TRIM(reorder_level)     LIKE '-%'
   OR TRIM(unit_price)        LIKE '-%';

-- Found: P019 qty=-300, P020 reorder=-50, P021 price=-180,
--         P022 qty=-100, P023 reorder=-30
```

Why this query?

LIKE '-%' is a simple string pattern check — it asks: 'does this value start with a minus sign?'
This is deliberately used instead of CAST(quantity_in_stock AS INT) < 0 because the column still contains mixed data including non-numeric text. CAST on 'abc' would throw an error in strict mode.

TRIM() is applied first to remove any leading whitespace that could hide a leading '-' character.
This is a safe, type-agnostic pattern that works regardless of the column's storage type.

3.5 Invalid Supplier Contact Numbers

Valid Indian mobile numbers are exactly 10 digits long and begin with a digit between 6 and 9. Numbers like '12345', '0000000000', 'abc987xyz', or negative values all fail this rule.

```
SELECT product_id, supplier_name, supplier_contact
FROM supply
WHERE supplier_contact IS NULL
   OR TRIM(supplier_contact) = ''
   OR NOT TRIM(supplier_contact) REGEXP '^[6-9][0-9]{9}$';

-- Found: P014 blank, P029 '12345', P030 'abc987xyz',
--         P031 '0', P032 '-9988776655', P078 '1234567890'
```

Why this query?

The regex '^[6-9][0-9]{9}$' encodes a specific business rule about Indian mobile numbers:

[6-9] first digit must be 6, 7, 8, or 9 (Indian telecom standard)

[0-9]{9} exactly 9 more digits must follow (total = 10 digits)
^ and $ anchors ensure no extra characters before or after

The three-part WHERE is intentional: NULL check first (short-circuit safety), then empty string, then the regex. Applying REGEXP to NULL returns NULL (not FALSE), so IS NULL must be checked first to capture those rows independently from the format-violation rows.

3.6 Expiry Date Earlier Than Restock Date

A product cannot logically expire before it was restocked. The challenge is that dates exist in three different formats in the same column, so they must be parsed before comparison.

```
SELECT product_id, product_name, last_restocked_date, expiry_date
FROM supply WHERE
  DATE_FORMAT(
    CASE
      WHEN expiry_date LIKE '____-__-__' THEN STR_TO_DATE(expiry_date,'%Y-
%m-%d')
      WHEN expiry_date LIKE '__-___-__'  THEN STR_TO_DATE(expiry_date,'%d-
%b-%y')
      WHEN expiry_date LIKE '__/__/____'  THEN
STR_TO_DATE(expiry_date,'%d/%m/%Y')
    END, '%Y-%m-%d')
  <
  DATE_FORMAT(
    CASE
      WHEN last_restocked_date LIKE '____-__-__' THEN
STR_TO_DATE(last_restocked_date,'%Y-%m-%d')
      WHEN last_restocked_date LIKE '__-___-__'  THEN
STR_TO_DATE(last_restocked_date,'%d-%b-%y')
      WHEN last_restocked_date LIKE '__/__/____'  THEN
STR_TO_DATE(last_restocked_date,'%d/%m/%Y')
    END, '%Y-%m-%d');

-- Found: P043 Paneer 200g, P044 Butter 100g, P045 Ghee 500ml, P046 Yogurt
400g
```

Why this query?

Three date formats coexist in the same column: YYYY-MM-DD, DD-Mon-YY (e.g. 03-Jan-24), and DD/MM/YYYY. The LIKE pattern uses underscores (_) as single-character wildcards to detect the format of each row, then STR_TO_DATE parses it with the matching format string.

DATE_FORMAT(..., '%Y-%m-%d') then normalises both dates to the same string before comparing, ensuring '2024-03-25' and '25-Mar-24' are compared as dates, not strings. String comparison of dates in mixed formats would yield completely wrong results.

Pattern to remember: detect format with LIKE -> parse with STR_TO_DATE -> normalise with DATE_FORMAT -> compare. This is the standard, reusable approach for mixed-format date columns.

3.7 Zero-Stock Products Marked as Active

A product with zero units in stock cannot be actively available for sale. These records represent a logical inconsistency between the status field and the inventory figure.

```
SELECT product_id, product_name, quantity_in_stock, status
FROM supply
WHERE TRIM(quantity_in_stock) = '0'
  AND LOWER(TRIM(status)) = 'active';

-- Found: P078, P079, P084 Vanilla Extract 50ml,
--         P085 Baking Powder 100g, P086 Baking Soda 200g
```

Why this query?

TRIM() removes surrounding whitespace before comparison — a stored '0 ' (with trailing space) would not equal '0' without it. LOWER() normalises status case so 'Active', 'ACTIVE', and 'active' all match — making the check case-insensitive and robust against inconsistent entry.

This is a business rule violation, not a format issue. The correct fix is an UPDATE (not DELETE) because the product may be real — only its status is logically wrong.

3.8 Category and Warehouse Value Audit

Categorical columns often accumulate variations of the same value due to inconsistent data entry. A distinct-value audit with counts reveals all variants before standardisation begins.

```
SELECT DISTINCT category, COUNT(*) AS cnt
FROM supply GROUP BY category ORDER BY cnt DESC;

SELECT DISTINCT warehouse_location, COUNT(*) AS cnt
FROM supply GROUP BY warehouse_location ORDER BY cnt DESC;
```

Category Variants Found	Warehouse Variants Found
dry fruits, DryFruits, dry-fruits -> Dry Fruits	wh-a -> Warehouse A
cereals, cereal -> Cereals	WH B -> Warehouse B
Grocery, groceries -> Grocery	Warehouse-C -> Warehouse C
Test, ???, NA -> (deleted)	???, unknown, NA -> (deleted)

Why this query?

GROUP BY + COUNT(*) + ORDER BY cnt DESC is the standard 'value frequency' pattern for auditing
categorical columns. Ordering by count descending naturally shows legitimate values first (high count)

and noise / garbage values last (low count), making it easy to identify what needs standardisation. Always run this before writing any standardisation logic — you cannot write the CASE statement without first knowing all the variants that exist in the data.

4. Data Cleaning Pipeline

With all issues fully documented from the audit, cleaning is applied in a deliberate order. The sequence matters: dummy rows are removed first so they do not interfere with later steps, then NULL fills, then duplicate removal, then type validation, and so on. Changing the order can produce different — and incorrect — results.

Key Concept

Cleaning Order Principle: Always apply deletions of clearly-invalid rows before updates, and fix structural issues (duplicates, nulls, invalid types) before applying business-rule checks (negative values, date logic, zero-stock status). This prevents a cleaned row from being re-contaminated by a step that should have run earlier.

4.1 Remove Fully Invalid / Dummy Rows

Rows with placeholder product names like 'TestProduct', 'Dummy Item', '???', 'NA', or 'unknown' represent test data or unfilled records. These are deleted first because they would otherwise be processed by all subsequent steps.

```
DELETE FROM supply
WHERE LOWER(TRIM(product_name)) IN ('testproduct','dummy
item','???','na','unknown');
```

Why this query?

LOWER(TRIM(...)) normalises the value before comparing it to the IN list. Without LOWER, the values 'Unknown', 'UNKNOWN', and 'unknown' would each need to be listed separately. Without TRIM,
a value stored as ' unknown ' (with spaces) would not match and would survive the delete.

Using IN is cleaner and faster than chaining multiple OR conditions. The database engine can use a hash lookup for IN lists, making it more efficient than repeated string comparisons.

4.2 Fill NULL / Blank Text Columns

Rather than deleting rows with missing text values, we fill them with sensible defaults. This preserves potentially valuable numeric data (stock levels, prices) that would otherwise be lost along with the row.

```
UPDATE supply SET
  product_name = CASE WHEN product_name IS NULL OR TRIM(product_name)=''
                 THEN 'Unknown Product' ELSE product_name END,
  category     = CASE WHEN category IS NULL OR TRIM(category)=''
                 THEN 'Unknown' ELSE category END,
  supplier_name = CASE WHEN supplier_name IS NULL OR TRIM(supplier_name)=''
                  THEN 'Unknown Supplier' ELSE supplier_name END,
  warehouse_location = CASE WHEN warehouse_location IS NULL OR
TRIM(warehouse_location)=''
                       THEN 'Warehouse A' ELSE warehouse_location END,
  batch_number = CASE WHEN batch_number IS NULL OR TRIM(batch_number)=''
                 THEN 'Not Available' ELSE batch_number END,
  status       = CASE WHEN status IS NULL OR TRIM(status)=''
                 THEN 'Active' ELSE status END;
```

Why this query?

All six text columns are updated in a single UPDATE statement — one table scan is far more efficient than six separate UPDATE statements, each requiring its own full scan of 105 rows.

The CASE WHEN ... ELSE column_name END pattern is key: the ELSE branch returns the original value
unchanged. Rows with already-valid values are untouched. Only truly NULL or blank rows receive the default. This is a safe, non-destructive update — it cannot accidentally overwrite good data.

4.3 Remove Duplicate product_id Records

Duplicates are removed by keeping the row with the lowest row_id (first row loaded) and deleting later copies. A temporary auto-increment column is added first to give each row a unique handle for the self-join.

```
-- Add a unique row number so each row has a handle
ALTER TABLE supply ADD COLUMN row_id INT AUTO_INCREMENT PRIMARY KEY;

-- Delete later copies, keep the earliest (lowest row_id)
DELETE t1
FROM supply t1
JOIN supply t2
  ON t1.product_id = t2.product_id
 AND t1.row_id > t2.row_id;
```

Why this query?

The self-join pattern works by pairing each row (t1) with every other row (t2) that shares the same product_id. The condition t1.row_id > t2.row_id identifies t1 as the 'later' copy.

DELETE t1 removes only the later copy, leaving the earlier one (t2) intact.

Without the row_id column, there is no reliable way to distinguish two otherwise-identical rows. Adding a surrogate key before deduplication is the standard, portable solution.

Alternative (MySQL 8+): ROW_NUMBER() OVER (PARTITION BY product_id ORDER BY row_id) in a CTE
achieves the same result with cleaner syntax, but the self-join works on all MySQL versions.

4.4 Delete Rows with Text in Numeric Columns

Rows where quantity_in_stock, unit_price, or reorder_level contain non-numeric text cannot be meaningfully cleaned — the correct number cannot be inferred from 'abc' or 'expensive'. These rows are deleted.

```
DELETE FROM supply
WHERE TRIM(quantity_in_stock) NOT REGEXP '^-?[0-9]+$'
   OR TRIM(unit_price)        NOT REGEXP '^-?[0-9]+$'
   OR TRIM(reorder_level)     NOT REGEXP '^-?[0-9]+$';
```

Why this query?

Note the regex is '^-?[0-9]+$' (with optional minus -?) rather than '^[0-9]+$' as used in the audit step. In cleaning we want to delete non-numeric values only — negative numbers (-300) are real numbers and are handled separately in the next step. Using '^-?[0-9]+$' preserves those rows.

The OR logic ensures any row with even one corrupted numeric field is removed entirely — a row with a valid quantity but an invalid price is still unusable for financial reporting.

4.5 Delete Rows with Negative Values

After confirming all remaining values are valid numbers, rows with negative quantities, reorder levels, or prices are removed as business rule violations.

```
DELETE FROM supply
WHERE TRIM(quantity_in_stock) LIKE '-%'
   OR TRIM(reorder_level)     LIKE '-%'
   OR TRIM(unit_price)        LIKE '-%';
```

Why this query?

LIKE '-%' is used again (same as the audit step) because the column is still VARCHAR at this stage. We could also write CAST(quantity_in_stock AS INT) < 0, but if any non-numeric value somehow survived the previous step, CAST would throw an error. LIKE '-%' is always type-safe.

Critically, this step runs AFTER step 4.4. If it ran before, text values like 'bad' would not match '-%' and would survive incorrectly. Order of operations matters in a cleaning pipeline.

4.6 Fix Zero-Stock Products Marked as Active

```
UPDATE supply
SET status = 'Inactive'
WHERE TRIM(quantity_in_stock) = '0'
  AND LOWER(TRIM(status)) = 'active';
```

Why this query?

This is a business rule correction, not a deletion. The product exists and its other data may be valid — only the status is logically wrong. Deleting the row would lose valid product metadata. An UPDATE is the correct operation whenever a value can be inferred or corrected rather than removed.

4.7 Standardize the STATUS Column

The status column contains 'Active', 'inactive', 'ACTIVE', 'pending', '???', and 'unknown'. Fixed in two steps: first proper-case all values, then delete any row whose status is still not one of the three allowed values.

```
-- Step 1: Proper-case all status values  (ACTIVE -> Active, inactive ->
Inactive)
UPDATE supply
SET status = CONCAT(
  UPPER(LEFT(LOWER(TRIM(status)), 1)),
  LOWER(SUBSTR(TRIM(status), 2))
);

-- Step 2: Remove rows with invalid / unrecognised status values
DELETE FROM supply
WHERE status NOT IN ('Active','Inactive','Pending');
```

Why this query?

The proper-casing formula breaks down as:

TRIM(status)	removes surrounding whitespace
LOWER(...)	lowercases everything first (handles 'ACTIVE' -> 'active')
LEFT(..., 1)	takes the first character
UPPER(...)	uppercases just that first character
SUBSTR(..., 2)	the rest of the string (already lowercased)
CONCAT(...)	joins them back: 'A' + 'ctive' = 'Active'

Two steps are necessary: after proper-casing, 'pending' becomes 'Pending' and is kept.
But '???' becomes '???' (unchanged) and is correctly deleted by the IN check in step 2.
Doing the DELETE before proper-casing would incorrectly remove valid 'inactive'/'PENDING' rows.

4.8 Standardize the CATEGORY Column

```
UPDATE supply SET category =
  CASE
    WHEN LOWER(REPLACE(TRIM(category),'-',' ')) LIKE '%dry fruit%' THEN 'Dry
Fruits'
    WHEN LOWER(TRIM(category)) IN ('cereals','cereal')              THEN
'Cereals'
    WHEN LOWER(TRIM(category)) IN ('grains','grain')                THEN
'Grains'
    WHEN LOWER(TRIM(category)) IN ('oils','oil')                    THEN
'Oils'
    WHEN LOWER(TRIM(category)) IN ('spices','spice')                THEN
'Spices'
    WHEN LOWER(TRIM(category)) IN ('pulses','pulse')                THEN
'Pulses'
    WHEN LOWER(TRIM(category)) IN ('grocery','groceries')           THEN
'Grocery'
    ELSE CONCAT(UPPER(LEFT(LOWER(TRIM(category)),1)),
LOWER(SUBSTR(TRIM(category),2)))
  END;
```

Why this query?

The CASE statement maps every known variant to its canonical form. Order matters:
- Dry Fruits is checked first using LIKE '%dry fruit%' after stripping hyphens with REPLACE. This handles 'dry fruits', 'DryFruits', and 'dry-fruits' in one condition.
- Other categories use IN lists for exact matching after LOWER/TRIM normalisation.
- The ELSE branch applies generic proper-casing for any category not in the explicit list, so new valid categories added in the future are handled without code changes.

Design pattern: explicit rules first (highest-variance cases), generic fallback last.

4.9 Standardize Warehouse Location

```
UPDATE supply SET warehouse_location =
  CASE
    WHEN LOWER(REPLACE(REPLACE(TRIM(warehouse_location),' ',''),'-',''))
LIKE '%a'
         THEN 'Warehouse A'
    WHEN LOWER(REPLACE(REPLACE(TRIM(warehouse_location),' ',''),'-',''))
LIKE '%b'
         THEN 'Warehouse B'
```

```
    WHEN LOWER(REPLACE(REPLACE(TRIM(warehouse_location),' ',''),'-',''))
LIKE '%c'
         THEN 'Warehouse C'
    ELSE NULL
  END;

DELETE FROM supply WHERE warehouse_location IS NULL;
```

Why this query?

Nested REPLACE strips both spaces and hyphens before LOWER is applied:
'Warehouse-C' -> strip spaces -> 'Warehouse-C' -> strip hyphens -> 'WarehouseC' -> lower -> 'warehousec'
'WH B' -> strip spaces -> 'WHB' -> strip hyphens -> 'WHB' -> lower -> 'whb'
Both then match LIKE '%b' / LIKE '%c' and are correctly standardised.

Unknown values like '???' or 'Warehouse X' map to NULL via the ELSE NULL branch.
A subsequent DELETE removes those rows since they cannot be assigned to any valid warehouse.
Clean two-step pattern: UPDATE to standardise, DELETE to eliminate the unresolvable.

4.10 Remove Special Characters from Name Columns

```
UPDATE supply SET
  product_name = TRIM(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(
    REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(product_name,

'@',''),'#',''),'!',''),'$',''),'%',''),'^',''),'&',''),'*',''),'?',''),'~',
''')),
  supplier_name = TRIM(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(
    REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(supplier_name,

'@',''),'#',''),'!',''),'$',''),'%',''),'^',''),'&',''),'*',''),'?',''),'~',
''));

-- Remove rows whose product_name now starts with a digit
DELETE FROM supply WHERE LEFT(TRIM(product_name),1) BETWEEN '0' AND '9';
```

Why this query?

MySQL before 8.0.4 lacks REGEXP_REPLACE, so special characters are stripped by chaining ten REPLACE calls. Each REPLACE returns a new string passed as input to the next — a functional composition pattern. The final TRIM() removes any leading/trailing spaces that remain after stripping.

The follow-up DELETE removes rows where product_name now starts with a digit (e.g. '123 Item'). Product names starting with digits are treated as invalid identifiers in this domain — they likely represent code fragments, corrupted data, or placeholder entries.

4.11 Fix Invalid Supplier Contacts

```
-- Step 1: NULL out every value that fails the Indian mobile format
UPDATE supply SET supplier_contact = NULL
WHERE supplier_contact IS NULL
   OR TRIM(supplier_contact) = ''
   OR NOT TRIM(supplier_contact) REGEXP '^[6-9][0-9]{9}$';

-- Step 2: Fill remaining NULLs with a placeholder
UPDATE supply SET supplier_contact = '9848786858'
WHERE supplier_contact IS NULL;
```

Why this query?

Two-step approach: first NULL out all invalid contacts (consolidates all bad values into one uniform NULL state), then fill NULLs with a placeholder in one clean UPDATE.
This is cleaner than trying to directly replace diverse invalid values ('abc987xyz', '12345', '0') with the placeholder in a single step — you would need to enumerate every failure mode.

Using a placeholder preserves all other product data (stock, price, category) for the row.
In production this would be flagged for human review rather than silently replacing the contact.

4.12 Standardize Date Formats to YYYY-MM-DD

Three date formats were found in both date columns. Each is converted separately using format-detection LIKE patterns, then invalid placeholder values are NULLed out, and rows with NULL dates are deleted.

```
-- Convert DD-Mon-YY  (e.g. '03-Jan-24')
UPDATE supply SET last_restocked_date =
  DATE_FORMAT(STR_TO_DATE(last_restocked_date,'%d-%b-%y'),'%Y-%m-%d')
WHERE last_restocked_date LIKE '__-___-__';

-- Convert DD/MM/YYYY  (e.g. '15/02/2024')
UPDATE supply SET last_restocked_date =
  DATE_FORMAT(STR_TO_DATE(last_restocked_date,'%d/%m/%Y'),'%Y-%m-%d')
WHERE last_restocked_date LIKE '__/__/____';

-- NULL out placeholder values
UPDATE supply SET last_restocked_date = NULL
WHERE last_restocked_date IN ('future_date','99-99-2024','00-00-
0000','NA','???')
   OR last_restocked_date LIKE '99%';

-- Same three steps repeated for expiry_date ...

-- Remove rows with unresolvable dates in either column
DELETE FROM supply WHERE expiry_date IS NULL OR last_restocked_date IS NULL;
```

Why this query?

STR_TO_DATE(value, format) parses a string into a MySQL DATE using format tokens:
%d = day (01-31), %b = month abbreviation (Jan-Dec), %y = 2-digit year, %Y = 4-digit year
DATE_FORMAT(date, '%Y-%m-%d') converts the parsed DATE back to the standard ISO string.

The LIKE '__-___-__' pattern uses underscores as character-position wildcards:
__ = exactly 2 chars (day), - literal, ___ = exactly 3 chars (month abbr), - literal, __ = 2-char year
This pattern uniquely identifies the DD-Mon-YY format.

Rows already in YYYY-MM-DD format match neither LIKE pattern and are skipped entirely.
The WHERE clause acts as a format filter, making each UPDATE safe and idempotent.
Running STR_TO_DATE on an already-correct date with the wrong format would corrupt it.

4.13 Delete Rows Where Expiry Date < Restock Date

After standardising all dates to YYYY-MM-DD, the temporal business rule is enforced: any product where the expiry date precedes the restock date is deleted.

```
DELETE FROM supply
WHERE STR_TO_DATE(expiry_date,'%Y-%m-%d')
    < STR_TO_DATE(last_restocked_date,'%Y-%m-%d');

-- Found and deleted: P043 Paneer 200g, P044 Butter 100g,
--                     P045 Ghee 500ml,  P046 Yogurt 400g
```

Why this query?

After date standardisation, a simple STR_TO_DATE comparison is sufficient. The complex CASE-based
multi-format parsing from the audit step is no longer needed — all remaining dates are in one format.
This demonstrates a key benefit of the cleaning pipeline: earlier steps simplify later ones.

These 4 records (P043-P046) represent real data entry errors — month and day were likely swapped during manual entry (e.g., expiry 2024-03-01 should have been 2024-01-03).

5. Creating the Clean Output Table

5.1 DDL — clean_inventory

A new table is created with proper data types and NOT NULL constraints. The constraints act as a final safety net — the subsequent INSERT will reject any row that still violates a constraint, preventing dirty data from entering the clean table.

```
CREATE TABLE clean_inventory (
  product_id          VARCHAR(10)  PRIMARY KEY,
  product_name        VARCHAR(100) NOT NULL,
  category            VARCHAR(50)  NOT NULL,
  supplier_name       VARCHAR(100) NOT NULL,
  supplier_contact    INT,                        -- nullable: placeholder
used if invalid
  warehouse_location  VARCHAR(20)  NOT NULL,
  quantity_in_stock   INT          NOT NULL,
  reorder_level       INT          NOT NULL,
  unit_price          INT          NOT NULL,
  last_restocked_date DATE         NOT NULL,
  expiry_date         DATE         NOT NULL,
  batch_number        VARCHAR(20)  NOT NULL,
  status              VARCHAR(20)  NOT NULL
);
```

Key Concept

Type enforcement in the DDL is what makes this a true data cleaning project — not just a query exercise. By using INT for numeric columns and DATE for date columns, the database engine itself will reject any value that is not a valid integer or date. NOT NULL constraints prevent any row with missing mandatory fields from being inserted. This is 'defence in depth' for data quality: the pipeline cleans the data AND the schema enforces it stays clean going forward.

5.2 Insert Validated Rows

The INSERT...SELECT statement transfers only rows that pass all final validation conditions. A final WHERE clause re-validates key business rules as a last checkpoint before committing to the clean table.

```
INSERT INTO clean_inventory (
  product_id, product_name, category, supplier_name, supplier_contact,
  warehouse_location, quantity_in_stock, reorder_level, unit_price,
  last_restocked_date, expiry_date, batch_number, status
)
SELECT
  product_id,
  TRIM(product_name),
  category,
  TRIM(supplier_name),
  CASE WHEN TRIM(supplier_contact) REGEXP '^[6-9][0-9]{9}$'
       THEN supplier_contact ELSE NULL END,
  warehouse_location, quantity_in_stock, reorder_level, unit_price,
  last_restocked_date, expiry_date, batch_number, status
FROM supply
WHERE
  product_name IS NOT NULL AND TRIM(product_name) != ''
  AND category IS NOT NULL
```

```
AND supplier_name IS NOT NULL
AND warehouse_location IN ('Warehouse A','Warehouse B','Warehouse C')
AND batch_number IS NOT NULL AND TRIM(batch_number) != ''
AND status IN ('Active','Inactive','Pending')
AND quantity_in_stock >= 0
AND reorder_level > 0;
```

Why this query?

The final WHERE clause is a defensive re-check, not redundant work. Even after all cleaning steps, an edge case could theoretically let a problematic row slip through. Re-validating here means clean_inventory is guaranteed clean by construction, not just by trust in the pipeline.

INSERT...SELECT is the standard pattern for creating a clean copy from a dirty working table.
It is non-destructive — the original supply table remains intact for audit and rollback purposes.
The TRIM() in the SELECT list and the CASE for supplier_contact are final safety applications.

6. Cleaning Summary

#	Issue Identified	Detection Query Pattern	Fix Applied
1	Dummy / placeholder rows	`LOWER(name) IN (list)`	DELETE
2	NULL / blank text fields	`IS NULL OR TRIM()=''`	UPDATE → default value
3	Duplicate product_id	`PARTITION BY / HAVING COUNT > 1`	DELETE keeping min row_id
4	Text in numeric columns	`NOT REGEXP '^-?[0-9]+$'`	DELETE
5	Negative numeric values	`LIKE '-%'`	DELETE
6	Zero-stock Active products	`qty=0 AND LOWER(status)=active`	UPDATE status → Inactive
7	Non-standard STATUS values	`NOT IN allowed set`	Proper-case + DELETE outliers
8	Category name variants	`DISTINCT category COUNT(*) audit`	UPDATE → canonical name
9	Warehouse name variants	`DISTINCT warehouse COUNT(*) audit`	UPDATE or DELETE
10	Special chars in names	`Chained REPLACE chain`	UPDATE stripping chars
11	Invalid supplier contacts	`NOT REGEXP '^[6-9][0-9]{9}$'`	NULL out → placeholder

#	Issue Identified	Detection Query Pattern	Fix Applied
12	Mixed date formats	`LIKE pattern + STR_TO_DATE`	Normalise to YYYY-MM-DD
13	Expiry date < Restock date	`DATE comparison after normalise`	DELETE

Raw Row Count	Rows Removed	Clean Row Count
105	**~33**	**~72**

7. Key SQL Concepts & Quick Reference

This section summarises every core SQL technique used in this project — useful for revision or interview preparation.

Concept / Function	What it Does & When to Use It
`REGEXP / NOT REGEXP`	Pattern matching using regular expressions. Use to validate formats — phone numbers, codes, pure-numeric strings. More powerful and readable than chained LIKE conditions.
`TRIM()`	Removes leading and trailing whitespace. Always apply before string comparisons to avoid false non-matches caused by invisible spaces.
`LOWER() / UPPER()`	Converts string to lower/upper case. Apply before comparing or grouping text to make checks case-insensitive.
`REPLACE(str, old, new)`	Replaces all occurrences of old with new. Chain multiple calls to strip multiple characters — each call returns a new string passed into the next.
`STR_TO_DATE(str, fmt)`	Parses a string into a MySQL DATE using the specified format tokens. Essential for converting text-stored dates into actual date values for comparison.
`DATE_FORMAT(date, fmt)`	Formats a DATE as a string. Use after STR_TO_DATE to normalise all dates into a consistent format like '%Y-%m-%d'.
`LIKE with _ wildcards`	_ matches exactly one character. Use 'LIKE __-___-__' to detect specific date formats by character position — simpler than a regex for fixed-width patterns.
`Window Functions`	COUNT(*) OVER (PARTITION BY col) computes a count per group while keeping all row data visible. Perfect for identifying duplicates without losing context.

Concept / Function	What it Does & When to Use It
`CASE WHEN...ELSE...END`	SQL's conditional expression. Use in SELECT for computed columns, in UPDATE for conditional row-by-row transformations, in ORDER BY for custom sort logic.
`INSERT...SELECT`	Inserts rows from one query result into another table. The standard pattern for creating a clean output table from a dirty working table — non-destructive.
`Self-Join for Dedup`	Joining a table to itself on a shared key. Combined with a surrogate row_id to identify and delete duplicate copies while keeping the earliest version.
`SUM(condition)`	In MySQL, booleans evaluate to 1/0. SUM(col IS NULL) counts NULL rows — running this across all columns in one SELECT creates an efficient null profile.

8. References & Further Reading

Data Quality & Cleaning Theory

- Rahm, E. & Do, H. H. (2000). Data Cleaning: Problems and Current Approaches. IEEE Data Engineering Bulletin, 23(4), 3-13. — The foundational academic survey of data quality problems, categorising issues into single-source vs. multi-source, schema-level vs. instance-level.
- Redman, T. C. (1996). Data Quality for the Information Age. Artech House. — Classic text establishing the business case for data quality management and defining quality dimensions: accuracy, completeness, consistency, timeliness.
- Wang, R. Y. & Strong, D. M. (1996). Beyond Accuracy: What Data Quality Means to Data Consumers. Journal of Management Information Systems, 12(4), 5-33. — Introduced the widely-cited four-category data quality framework: intrinsic, contextual, representational, and accessibility quality.
- Loshin, D. (2010). The Practitioner's Guide to Data Quality Improvement. Morgan Kaufmann. — Practical framework for implementing data quality processes in enterprise data systems, with case studies.

MySQL Technical References

- MySQL 8.0 Reference Manual — String Functions. Oracle Corporation. dev.mysql.com/doc/refman/8.0/en/string-functions.html — Covers TRIM, REPLACE, CONCAT, LEFT, SUBSTR, and all functions used in this project.

- MySQL 8.0 Reference Manual — Regular Expressions. Oracle Corporation. dev.mysql.com/doc/refman/8.0/en/regexp.html — Explains REGEXP, NOT REGEXP, and supported regex syntax in MySQL.
- MySQL 8.0 Reference Manual — Date and Time Functions. Oracle Corporation. dev.mysql.com/doc/refman/8.0/en/date-and-time-functions.html — Covers STR_TO_DATE, DATE_FORMAT, DATEDIFF, and related functions.
- MySQL 8.0 Reference Manual — Window Functions. Oracle Corporation. dev.mysql.com/doc/refman/8.0/en/window-functions.html — Explains OVER, PARTITION BY, ROW_NUMBER, RANK, and COUNT used in duplicate detection.
- Forta, B. (2013). MySQL Crash Course. Sams Publishing. — Accessible practical guide to MySQL covering all SQL operations and patterns used throughout this project.

Supply Chain Data Management

- Chopra, S. & Meindl, P. (2015). Supply Chain Management: Strategy, Planning, and Operation (6th ed.). Pearson. — Establishes why data integrity is critical to demand forecasting, inventory optimisation, and supplier management.
- Lambert, D. M. (2008). Supply Chain Management: Processes, Partnerships, Performance. Supply Chain Management Institute. — Discusses how data quality failures propagate across supply chain tiers and impact operational decisions.

Project Repository

- SQL source code and raw dataset: github.com/tensakuro/mysql-files
- Clone command: git clone https://github.com/tensakuro/mysql-files.git